墨韵（MoYun）AI小说创作系统 - 完整使用指南

> **文档版本**: v1.2.0
> **更新日期**: 2026-04-21
> **技术栈**: Spring Boot 3.2 + Vue 3 + LangChain4j 0.35.0

目录

- 一、项目概述
- 二、技术栈
- 三、项目结构
- 四、数据库设计
- 五、核心功能详解
- 六、AI Agent系统
- 七、文风卡片系统
- 八、向量检索系统
- 九、LLM配置系统
- 十、部署指南

一、项目概述

1.1 核心定位

墨韵 是一个基于AI的百万字长篇小说创作系统，通过多Agent协作帮助用户自动生成小说内容。

1.2 目标用户

- 网络小说作者
- 创意写作爱好者
- AI辅助创作研究者

1.3 核心价值

- AI辅助长篇小说创作
- 多Agent协作（大纲→写作→摘要）
- 三线节奏系统（Quest主线/Fire感情线/Constellation世界观线）
- 文风克隆与复用（文风卡片系统）
- 伏笔管理与回收
- 人物关系图谱

二、技术栈

2.1 后端技术栈

类别	技术	版本	说明
框架	Spring Boot	3.2.0	主框架
语言	Java	17	LTS版本
AI框架	LangChain4j	0.35.0	AI Agent核心
ORM	MyBatis-Plus	3.5.5	数据库操作
数据库	MySQL	8.0+	主数据存储
文档数据库	MongoDB	6.0+	图谱/摘要/记忆
缓存	Redis	7.0+	缓存+向量搜索
消息队列	RabbitMQ	3.12+	异步章节生成
安全	Spring Security 6	-	认证授权
JWT	JJWT	0.12.5	Token管理

2.2 前端技术栈

类别	技术	版本	说明
框架	Vue	3.4+	SPA框架
语言	TypeScript	5.3+	类型安全
UI库	Element Plus	2.4+	组件库
状态管理	Pinia	2.1+	状态管理
HTTP	Axios	1.6+	HTTP客户端
图表	ECharts	5.4+	数据可视化

2.3 AI模型支持

提供商	模型	用途
阿里云 DashScope	qwen-turbo, qwen-plus, qwen-max	主要LLM
OpenAI	GPT-3.5, GPT-4	可选LLM
通义千问	text-embedding-v2	向量嵌入

三、项目结构

3.1 后端包结构

```
com.novel.system/
├── NovelSystemApplication.java          # 启动类
├── ai/                                  # AI模块
│   ├── agent/                           # Agent服务
│   │   ├── OutlineAgent.java            # 大纲Agent接口
│   │   ├── WritingAgent.java            # 写作Agent接口
│   │   ├── SummaryAgent*.java           # 摘要Agent
│   │   └── ArcSummaryAgent*.java        # 弧段摘要Agent
│   ├── chain/                           # 链式编排
│   │   ├── ChapterGenerationChain.java  # 章节生成链
│   │   ├── ValidationChain.java         # 验证链
│   │   └── HookDetectionService.java    # 钩子检测
│   ├── memory/                          # 记忆管理
│   │   ├── NovelChatMemory.java         # 对话记忆
│   │   ├── LongTermMemory.java          # 长期记忆
│   │   └── CharacterGraphService.java   # 人物关系图
│   ├── rag/                             # RAG检索
│   │   ├── RagService.java              # RAG服务
│   │   ├── RedisVectorService.java      # Redis向量
│   │   └── EmbeddingService.java        # 嵌入服务
│   └── tool/                            # Agent工具
│       ├── NovelToolProvider.java       # 工具提供器
│       └── CharacterTool.java           # 人物工具
├── config/                              # 配置类
│   ├── SecurityConfig.java              # 安全配置
│   ├── LangChainConfig.java             # LangChain配置
│   ├── JwtConfig.java                   # JWT配置
│   └── AuthFilter.java                  # 认证过滤器
├── controller/                          # 控制器
│   ├── AuthController.java              # 认证
│   ├── NovelController.java             # 小说
│   ├── ChapterController.java           # 章节
│   ├── CharacterController.java         # 人物
│   ├── ClueController.java              # 伏笔
│   ├── StyleCardController.java         # 文风卡片
│   ├── LlmConfigController.java         # LLM配置
│   └── ...
├── service/                             # 服务层
│   ├── chapter/                         # 章节服务
│   ├── character/                       # 人物服务
│   ├── clue/                            # 伏笔服务
│   ├── llm/                             # LLM服务
│   ├── style/                           # 文风服务
│   └── user/                            # 用户服务
├── entity/                              # 实体类
│   ├── mysql/                           # MySQL实体
│   └── mongo/                           # MongoDB实体
├── mapper/                              # Mapper接口
└── dto/                                 # 数据传输对象
    ├── request/                         # 请求DTO
    ├── response/                        # 响应DTO
```

```
    └── style/                          # 文风相关DTO
```

3.2 前端结构

```
frontend/src/
├── api/                                # API接口
│   ├── index.ts                        # 主API（含auth/novel/chapter等）
│   └── style.ts                        # 文风卡片API
├── components/                         # 组件
│   ├── Layout.vue                      # 布局组件
│   ├── StyleSelector.vue               # 文风选择器
│   └── ...
├── views/                              # 页面
│   ├── style/
│   │   └── StyleCardLibrary.vue        # 文风卡片库
│   ├── CreateNovel.vue                 # 创建小说
│   ├── NovelDetail.vue                 # 小说详情
│   └── ...
├── stores/                             # Pinia状态
├── types/                              # TypeScript类型
├── router/                             # 路由配置
└── utils/                              # 工具函数
```

四、数据库设计

4.1 MySQL表结构

4.1.1 用户相关

```sql
-- 用户表
sys_user (
    user_id         VARCHAR(32)     -- 用户ID
    username        VARCHAR(50)     -- 用户名
    password        VARCHAR(100)    -- 密码（BCrypt加密）
    email           VARCHAR(100)    -- 邮箱
    role            VARCHAR(20)     -- 角色：user/admin
    status          INT             -- 状态：0禁用/1正常
    create_time     DATETIME
)
```

4.1.2 小说相关

```sql
-- 小说主表
novel_base (
    novel_id            VARCHAR(32)     -- 小说ID
    user_id             VARCHAR(32)     -- 作者ID
    novel_name          VARCHAR(100)    -- 小说名称
    genre               VARCHAR(50)     -- 题材类型
    total_outline       TEXT            -- 总纲
    world_setting       TEXT            -- 世界观设定
    story_momentum      TEXT            -- 故事势能
    status              INT             -- 状态
```

```
    create_time         DATETIME
)

-- 卷表
novel_arc (
    arc_id              VARCHAR(32)     -- 卷ID
    novel_id            VARCHAR(32)     -- 小说ID
    arc_name            VARCHAR(100)    -- 卷名
    arc_outline         TEXT            -- 卷纲
    arc_order           INT             -- 卷序号
)

-- 章节表
novel_chapter (
    chapter_id          VARCHAR(32)     -- 章节ID
    novel_id            VARCHAR(32)     -- 小说ID
    chapter_num         INT             -- 章节号
    chapter_title       VARCHAR(200)    -- 章节标题
    content             LONGTEXT        -- 正文内容
    chapter_outline     TEXT            -- 章纲
    hook                TEXT            -- 结尾钩子
    word_count          INT             -- 字数
    synopsis_id         VARCHAR(32)     -- 章节简介ID
    is_deleted          INT             -- 软删除标记
    create_time         DATETIME
)

-- 弧段摘要表
arc_summary (
    arc_id              VARCHAR(32)
    novel_id            VARCHAR(32)
    chapter_from        INT             -- 起始章节
    chapter_to          INT             -- 结束章节
    summary             TEXT            -- 摘要内容
)
```

4.1.3 人物相关

```
-- 人物表
novel_character (
    character_id        VARCHAR(32)
    novel_id            VARCHAR(32)
    character_name      VARCHAR(50)     -- 姓名
    role                VARCHAR(20)     -- 角色类型:
protagonist/antagonist/supporting
    personality         TEXT            -- 性格设定
    background          TEXT            -- 背景故事
    status              VARCHAR(20)     -- 状态：alive/dead/unknown
    first_appear_chapter INT            -- 首次出场章节
)

-- 人物成长节点
character_growth (
    growth_id           VARCHAR(32)
    character_id        VARCHAR(32)
```

```
    chapter_num         INT
    growth_type         VARCHAR(50)     -- 成长类型
    description         TEXT
    create_time         DATETIME
)
```

4.1.4 伏笔/线索相关

```
-- 伏笔表
clue_book (
    clue_id             VARCHAR(32)
    novel_id            VARCHAR(32)
    clue_content        TEXT            -- 伏笔内容
    bury_chapter        INT             -- 埋下章节
    deadline            INT             -- 回收截止章节
    recycle_chapter     INT             -- 实际回收章节
    status              VARCHAR(20)     -- pending/recycled/expired
    create_time         DATETIME
)
```

4.1.5 文风卡片相关

```
-- 文风卡片主表
style_card (
    card_id             VARCHAR(32)
    name                VARCHAR(100)    -- 卡片名称
    source_novel        VARCHAR(200)    -- 来源小说
    created_by          VARCHAR(32)     -- 创建者

    rhythm_rules        JSON            -- 节奏规则
    dialogue_rules      JSON            -- 对话规则
    description_rules   JSON            -- 描写规则
    emotion_rules       JSON            -- 情绪规则
    forbidden_rules     JSON            -- 禁用规则
    preferred_rules     JSON            -- 偏好规则

    style_description   TEXT            -- 风格描述
    generation_template TEXT            -- 生成模板
    samples_json        JSON            -- 原文样本
    stats_json          JSON            -- 统计数据

    tags                VARCHAR(500)    -- 标签
    is_public           BOOLEAN         -- 是否公开
    usage_count         INT             -- 使用次数
    avg_rating          DECIMAL         -- 平均评分
    status              INT             -- 0分析中/1可用/2失败
    create_time         DATETIME
)

-- 文风样本表
style_sample (
    sample_id           VARCHAR(32)
    card_id             VARCHAR(32)
    sample_type         VARCHAR(50)     -- 悬疑/对话/战斗/心理/描写
```

```
    content             TEXT            -- 原文片段
    annotation          VARCHAR(500)    -- 风格注释
)

-- 用户文风卡片库
user_style_library (
    id                  VARCHAR(32)
    user_id             VARCHAR(32)
    card_id             VARCHAR(32)
    is_favorite         BOOLEAN
)

-- 小说文风绑定
novel_style_binding (
    id                  VARCHAR(32)
    novel_id            VARCHAR(32)
    card_id             VARCHAR(32)
    bind_time           DATETIME
)
```

4.1.6 LLM配置相关

```
-- LLM配置表
llm_config (
    config_id           VARCHAR(32)
    user_id             VARCHAR(32)
    name                VARCHAR(100)    -- 配置名称
    provider            VARCHAR(50)     -- 提供商：openai/dashscope/ollama
    base_url            VARCHAR(500)    -- API地址
    api_key             VARCHAR(500)    -- API密钥
    model_name          VARCHAR(100)    -- 模型名称
    max_tokens          INT             -- 最大token数
    temperature         DECIMAL         -- 温度参数
    usage_type          VARCHAR(50)     -- 用途：outline/writing/summary/all
    is_active           BOOLEAN         -- 是否激活
    priority            INT             -- 优先级
    create_time         DATETIME
)
```

4.2 MongoDB集合

```
// 章节摘要集合
chapter_summary {
    _id: ObjectId,
    novelId: String,
    chapterNum: Number,
    content: String,              // 摘要内容
    coreEvents: Array,            // 核心事件
    fingerprint: String,          // 指纹（一句话描述）
    hookLevel: String,            // 钩子强度
    createTime: Date
}

// 人物关系图谱
```

```
character_graph {
    _id: ObjectId,
    novelId: String,
    nodes: Array,                  // 节点：[{id, name, role, attributes}]
    edges: Array,                  // 边：[{source, target, relation, weight}]
    updateTime: Date
}

// 长期记忆
long_term_memory {
    _id: ObjectId,
    novelId: String,
    memoryType: String,            // 类型：character/plot/world
    content: String,
    importance: Number,            // 重要性评分
    relatedChapters: Array,        // 关联章节
    createTime: Date
}

// RAG向量文档
rag_document {
    _id: ObjectId,
    novelId: String,
    chunkIndex: Number,
    content: String,
    embedding: Array,              // 向量（存Redis时可为空）
    metadata: Object,
    createTime: Date
}
```

4.3 Redis数据结构

```
# RAG向量索引
FT.CREATE novel_summary_idx
    ON HASH PREFIX 1 "novel:summary:"
    SCHEMA
        novelId TAG
        content TEXT
        embedding VECTOR FLAT 6 TYPE FLOAT32 DIM 1536 DISTANCE_METRIC COSINE

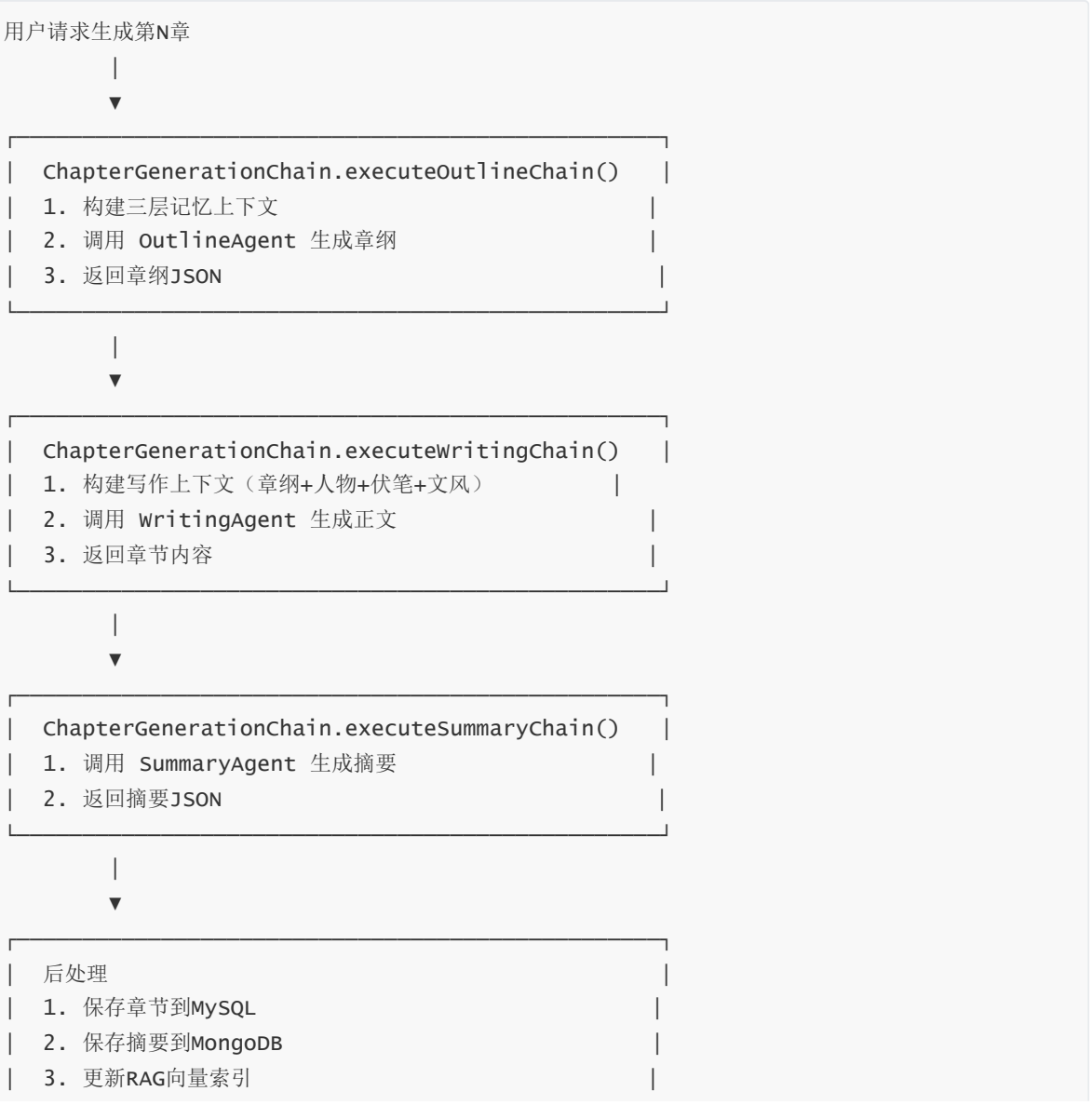
# 向量数据
HSET novel:summary:{novelId}:{chapterNum}
    novelId "{novelId}"
    content "{summary}"
    embedding "{vector}"
```

五、核心功能详解

5.1 小说创建流程



5.2 章节生成流程



```
|  4. 更新弧段摘要                                          |
|  5. 检测并更新伏笔状态                                    |
└──────────────────────────────────────────────────────┘
```

5.3 三层记忆体系

```
┌──────────────────────────────────────────────────────┐
|  第一层：弧段摘要（全局历史记忆）                          |
|  - 每50章生成一次弧段摘要                                 |
|  - 存储在MySQL arc_summary表                            |
|  - 防止AI对早期情节失忆                                   |
└──────────────────────────────────────────────────────┘
                          |
                          ▼
┌──────────────────────────────────────────────────────┐
|  第二层：近N章大纲（微观衔接）                            |
|  - 查询最近8章的章纲和钩子                                |
|  - 确保章节间衔接自然                                     |
└──────────────────────────────────────────────────────┘
                          |
                          ▼
┌──────────────────────────────────────────────────────┐
|  第三层：上一章钩子（本章承接）                           |
|  - 上一章结尾的悬念/钩子                                  |
|  - 本章开头必须直接承接                                   |
└──────────────────────────────────────────────────────┘
```

六、AI Agent系统

6.1 Agent架构

墨韵采用 **LangChain4j AI Services** 实现Agent系统：

```
// Agent接口定义（使用注解）
public interface WritingAgent {
    @SystemMessage("""
        你是一位专业的网络小说作家...
        【写作铁律】...
        【防幻觉三定律】...
        """)
    String generateChapter(@UserMessage String userPrompt);
}

// Agent实例创建
WritingAgent agent = AiServices.builder(WritingAgent.class)
    .chatLanguageModel(model)
    .tools(toolProvider.getAllTools())  // 注入工具
    .build();
```

6.2 三大核心Agent

Agent	职责	输入	输出
OutlineAgent	生成章纲	总纲+卷纲+三层记忆+人物+伏笔	章纲JSON
WritingAgent	生成正文	章纲+上下文+文风卡片	章节内容
SummaryAgent	生成摘要	章节内容	摘要JSON

6.3 Agent工具系统

Agent可以调用工具查询小说数据：

```
@Tool("查询人物设定")
public CharacterInfo getCharacterInfo(String characterName) {
    // 返回人物信息
}

@Tool("查询未回收伏笔")
public List<ClueInfo> getPendingClues(String novelId) {
    // 返回伏笔列表
}
```

6.4 防幻觉三定律

```
定律一：大纲即法律 — 严格遵循章纲内容
定律二：设定即物理 — 人物言行必须符合设定
定律三：发明需识别 — 创造新元素必须标注[新设定：XXX]
```

七、文风卡片系统

7.1 核心理念

不要让AI"学会风格"，让AI"戴着原文的镣铐跳舞"。

文风卡片是一个**可复用的风格资产**，包含：

- 结构化规则（机器可读，用于自动检查）
- 自然语言描述 + 原文片段（注入prompt，引导模型）

7.2 分析流程

```
用户上传小说文本（支持几十万字）
                  |
                  ▼
┌──────────────────────────────────────┐
|  程序化分析（零LLM成本）              |
|  - 句长分布                          |
|  - 段落分布                          |
|  - 对话比例                          |
|  - 感官词密度                        |
```

```
|  - 标点习惯                                  |
└──────────────────────────────────────────┘
                    |
                    ▼
┌──────────────────────────────────────────┐
|  自适应文本采样                              |
|  - 短篇（≤10章）：全采                      |
|  - 中篇（11-50章）：采15章×2000字           |
|  - 长篇（51-200章）：采20章×2000字          |
|  - 超长篇（200+章）：采30章×2000字          |
└──────────────────────────────────────────┘
                    |
                    ▼
┌──────────────────────────────────────────┐
|  LLM分析（1次调用）                          |
|  输入：统计数据 + 采样文本                   |
|  输出：完整文风卡片JSON                      |
└──────────────────────────────────────────┘
                    |
                    ▼
          存储到数据库，可复用
```

7.3 卡片内容示例

```yaml
name: "武动乾坤风格"
sourceNovel: "武动乾坤"

# 程序化统计数据
stats:
  avgSentenceLength: 53.8
  dialogueRatio: 0.31
  visualDensity: 2.5

# LLM分析结果
rhythm:
  description: "中长句铺陈为主，逗号高频串联动作"
  rule: "单句平均45-55字，关键转折处用短句"

dialogue:
  style: "功能驱动型，对话直接服务于剧情"
  quirks: ["统一用'道'作引导词", "对话后必跟神态描写"]

forbidden:
  words: ["不禁", "似乎", "嘴角微微上扬"]  # 这个作者不会用的词

samples:
  - type: "战斗场景"
    text: "林动望着那..."
    annotation: "短句堆叠制造紧张感"
```

7.4 使用方式

创建卡片：

1. 点击「文风卡片库」→「创建新卡片」
2. 粘贴文本或上传txt文件
3. 等待分析完成

使用卡片：

1. 创建小说时选择「写作风格」下拉框
2. 选择已创建的文风卡片
3. 生成章节时自动应用该风格

八、向量检索系统

8.1 RAG架构

```
章节生成完成
      |
      ▼
┌─────────────────────┐
|  生成章节摘要        |
└─────────┬───────────┘
          |
          ▼
┌─────────────────────┐
|  调用Embedding服务   | → text-embedding-v2
|  生成向量（1536维）  |
└─────────┬───────────┘
          |
          ▼
┌─────────────────────┐
|  存储到Redis         | → HSET novel:summary:{id}
|  更新向量索引        |
└─────────────────────┘
```

8.2 检索流程

```
生成新章节时
      |
      ▼
┌─────────────────────┐
|  用章纲作为查询      |
└─────────┬───────────┘
          |
          ▼
┌─────────────────────┐
|  生成查询向量        |
└─────────┬───────────┘
          |
          ▼
```

```
┌──────────────────────┐
│  Redis向量相似度搜索  │ → KNN查询，返回Top3
└──────────┬───────────┘
           │
           ▼
┌──────────────────────┐
│  注入到写作上下文     │
└──────────────────────┘
```

8.3 向量配置

```yaml
redis-stack:
  vector:
    index-name: novel_summary_idx
    dimension: 1536              # text-embedding-v2维度
    distance-metric: cosine      # 余弦相似度
```

九、LLM配置系统

9.1 多模型支持

系统支持配置多个LLM，按用途分配：

用途	说明	推荐模型
outline	生成章纲	qwen-turbo（快速）
writing	生成正文	qwen-plus（高质量）
summary	生成摘要	qwen-turbo（快速）
audit	审核/分析	qwen-plus
all	通用	任意模型

9.2 Fallback机制

```java
// 主模型失败时自动切换备用模型
public String executeWithFallback(String llmConfigId, Function<...> task) {
    try {
        // 尝试主模型
        return task.apply(primaryModel);
    } catch (Exception e) {
        // 失败后尝试备用模型
        return task.apply(fallbackModel);
    }
}
```

9.3 前端配置

访问「模型配置」页面，添加LLM配置：

- 配置名称
- API提供商（OpenAI/DashScope/Ollama）
- API地址
- API密钥
- 模型名称
- 用途类型

十、部署指南

10.1 环境要求

- JDK 17+
- MySQL 8.0+
- MongoDB 6.0+
- Redis Stack 7.0+（含向量搜索）
- RabbitMQ 3.12+（可选）

10.2 后端部署

```
# 1. 克隆项目
git clone <repo-url>

# 2. 配置环境变量
export MYSQL_HOST=localhost
export MYSQL_PORT=3306
export MYSQL_DATABASE=moyun_novel_system
export MYSQL_USERNAME=root
export MYSQL_PASSWORD=123456
export JWT_SECRET=your_jwt_secret_here

# 3. 创建数据库并执行SQL脚本
mysql -u root -p < backend/src/main/resources/sql/*.sql

# 4. 编译打包
cd backend
./mvnw clean package -DskipTests

# 5. 运行
java -jar target/ai-novel-system-1.0.0.jar
```

10.3 前端部署

```
cd frontend
npm install
npm run dev         # 开发模式
npm run build       # 生产构建
```

10.4 Docker部署

```
docker run -d --name novel-rabbitmq \
    --restart=always \
    -p 5672:5672 -p 15672:15672 \
    rabbitmq:3-management
```

附录

A. API接口列表

方法	路径	说明
POST	/auth/login	用户登录
POST	/auth/register	用户注册
GET	/novel/list	获取小说列表
POST	/novel	创建小说
POST	/novel/{id}/chapter	生成章节
GET	/novel/{id}/chapter/{num}	获取章节
POST	/style-card/create	创建文风卡片
GET	/style-card/list	获取卡片列表
POST	/style-card/bind	绑定文风卡片
GET	/llm-config/list	获取LLM配置
POST	/llm-config	添加LLM配置

B. 状态码说明

状态码	含义
200	成功
401	未登录或Token过期
403	无权限
404	资源不存在

状态码	含义
500	服务器错误

文档结束